

REFFI DWINO IGMAYODA (23)

254107060003

Sistem Informasi Bisnis

JOBSHEET 6

2.1 Percobaan 1 : Operasi Dasar Queue

1. queue23.java

```
public class queue23 {  
    int[] data;  
    int front;  
    int rear;  
    int size;  
    int max;  
  
    public queue23(int n) {  
        max = n;  
        data = new int[max];  
        size = 0;  
        front = rear = -1;  
    }  
  
    public boolean isEmpty() {  
        return size == 0;  
    }  
  
    public boolean isFull() {  
        return size == max;  
    }  
  
    public void peek() {  
        if (!isEmpty()) {  
            System.out.println("Elemen terdepan : " + data[front]);  
        } else {  
            System.out.println("Queue masih kosong");  
        }  
    }  
  
    public void print() {  
        if (isEmpty()) {  
            System.out.println("Queue masih kosong");  
        } else {  
            int i = front;  
            while (i != rear) {  
                System.out.print(data[i] + " ");  
                i = (i + 1) % max;  
            }  
            System.out.print(data[i]);  
        }  
    }  
}
```

2. queueMain23.java

```
import java.util.Scanner;  
  
public class queueMain23 {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Masukkan kapasitas queue: ");  
        int n = sc.nextInt();  
        queue23 Q = new queue23(n);  
  
        int pilih;  
        do {  
            System.out.println("\n=== MENU QUEUE ===");  
            System.out.println("1. Enqueue (Tambah data);");  
            System.out.println("2. Dequeue (Hapus data);");  
            System.out.println("3. Print (Cetak queue);");  
            System.out.println("4. Peek (lihat depan);");  
            System.out.println("5. Clear (Kosongkan queue);");  
            System.out.println("0. Keluar");  
            System.out.print("Pilih operasi: ");  
            pilih = sc.nextInt();  
  
            switch (pilih) {  
                case 1:  
                    System.out.print("Masukkan data baru: ");  
                    int dataMasuk = sc.nextInt();  
                    Q.enqueue(dataMasuk);  
                    break;  
                case 2:  
                    int dataKeluar = Q.dequeue();  
                    if (dataKeluar != -1) {  
                        System.out.println("Data yang dikeluarkan: " + dataKeluar);  
                    }  
                    break;  
                case 3:  
                    Q.print();  
                    break;  
                case 4:  
                    Q.peek();  
                    break;  
                case 5:  
                    Q.clear();  
                    break;  
                case 0:  
                    break;  
            }  
        } while (pilih != 0);  
    }  
}
```

Hasilnya

```
=== MENU QUEUE ===
1. Enqueue (Tambah data)
2. Dequeue (Hapus data)
3. Print (Cetak queue)
4. Peek (Lihat depan)
5. Clear (Kosongkan queue)
0. Keluar
Pilih operasi: 1
Masukkan data baru: 31

=== MENU QUEUE ===
1. Enqueue (Tambah data)
2. Dequeue (Hapus data)
3. Print (Cetak queue)
4. Peek (Lihat depan)
5. Clear (Kosongkan queue)
0. Keluar
Pilih operasi: 4
Elemen terdepan : 15

=== MENU QUEUE ===
1. Enqueue (Tambah data)
2. Dequeue (Hapus data)
3. Print (Cetak queue)
4. Peek (Lihat depan)
5. Clear (Kosongkan queue)
0. Keluar
Pilih operasi: █
```

2.1.3. Pertanyaan

1. Pada konstruktor, mengapa nilai awal atribut front dan rear bernilai -1, sementara atribut size bernilai 0?
 - front = -1 dan rear = -1 digunakan sebagai penanda bahwa queue masih kosong (belum ada data sama sekali).
 - size = 0 menunjukkan jumlah elemen memang belum ada.

Jadi:

- -1 → posisi belum valid (belum menunjuk index array)
- 0 → jumlah elemen kosong

2. Pada method Enqueue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (rear == max - 1) {
    rear = 0;
}
```

Mengecek apakah posisi rear sudah berada di index terakhir array (max - 1).

Jika iya, maka rear dikembalikan ke index 0.

3. Pada method Dequeue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (front == max - 1) {
    front = 0;
}
```

Mengecek apakah posisi front sudah di index terakhir array.

Jika iya, maka front dikembalikan ke index 0.

4. Pada method print, mengapa pada proses perulangan variabel i tidak dimulai dari 0 (int i=0), melainkan int i=front?

Data queue tidak selalu dimulai dari index 0 (akibat dequeue & circular).

front adalah posisi awal data yang valid.

Kalau mulai dari 0 → bisa mencetak data lama / tidak valid.

5. Perhatikan kembali method print, jelaskan maksud dari potongan kode berikut!

$i = (i + 1) \% \text{max};$

Menambahkan nilai i satu langkah ke depan (i + 1)

Lalu di-modulo dengan max agar tidak melebihi batas array

6. Tunjukkan potongan kode program yang merupakan queue overflow!

```
if (isFull()) {
```

```
    System.out.println("Queue penuh, tidak bisa enqueue");
```

```
}
```

7. Pada saat terjadi queue overflow dan queue underflow, program tersebut tetap dapat berjalan dan hanya menampilkan teks informasi. Lakukan modifikasi program sehingga pada saat terjadi queue overflow dan queue underflow, program dihentikan!

```
public void enqueue(int dt) {
    if (isFull()) {
        System.out.println(x: "QUEUE OVERFLOW! Queue sudah penuh.");
        System.out.println(x: "Program dihentikan.");
        System.exit(status: 1); // ← TAMBAHAN: hentikan program dengan kode error
    } else {
        if (isEmpty()) {
            front = rear = 0;
        } else {
            rear = (rear + 1) % max;
        }
        data[rear] = dt;
        size++;
    }
}

public int dequeue() {
    if (isEmpty()) {
        System.out.println(x: "QUEUE UNDERFLOW! Queue sudah kosong.");
        System.out.println(x: "Program dihentikan.");
        System.exit(status: 1); // ← TAMBAHAN: hentikan program dengan kode error
        return -1; // tidak akan pernah tercapai, tapi wajib ada
    } else {
        int dt = data[front];
        size--;
        if (isEmpty()) {
            front = rear = -1;
        } else {
            front = (front + 1) % max;
        }
        return dt;
    }
}
```

2.2. Percobaan 2 : Antrian Layanan Akademik

1. AntrianLayanan23.java

```
public class AntrianLayanan23 {
    Mahasiswa23[] data;
    int front;
    int rear;
    int size;
    int max;

    public AntrianLayanan23(int n) {
        max = n;
        data = new Mahasiswa23[max];
        size = 0;
        front = 0;
        rear = -1;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public boolean isFull() {
        return size == max;
    }

    public void lihatTerdepan() {
        if (isEmpty()) {
            System.out.println(x: "Antrian Kosong.");
        } else {
            System.out.println(x: "Mahasiswa Terdepan :");
            System.out.println(x: "NIM - NAMA - PRODI - KELAS");
            data[front].tampilkanData();
        }
    }

    public void print() {
        if (isEmpty()) {
            System.out.println(x: "Queue masih kosong");
        } else {
            int i = front;
            int count = 0;
        }
    }
}
```

2. LayananAkademikSIKAD23.java

```
import java.util.Scanner;

public class LayananAkademikSIKAD23 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianLayanan23 antrian = new AntrianLayanan23(n: 5);
        int pilihan;

        do {
            System.out.println(x: "\n=== Menu Antrian Layanan Akademik ===");
            System.out.println(x: "1. Tambah Mahasiswa ke Antrian");
            System.out.println(x: "2. Layanan Mahasiswa");
            System.out.println(x: "3. Lihat Mahasiswa Terdepan");
            System.out.println(x: "4. Lihat Semua Antrian");
            System.out.println(x: "5. Jumlah Mahasiswa Dalam Antrian");
            System.out.println(x: "0. Keluar");
            System.out.print(s: "Pilih operasi: ");
            pilihan = sc.nextInt();
            sc.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.print(s: "NIM : ");
                    String nim = sc.nextLine();
                    System.out.print(s: "Nama : ");
                    String nama = sc.nextLine();
                    System.out.print(s: "Prodi : ");
                    String prodi = sc.nextLine();
                    System.out.print(s: "Kelas : ");
                    String kelas = sc.nextLine();
                    Mahasiswa23 mhs = new Mahasiswa23(nim, nama, prodi, kelas);
                    antrian.tambahAntrian(mhs);
                    break;
                case 2:
                    Mahasiswa23 dilayani = antrian.layaniMahasiswa();
                    if (dilayani != null) {
                        System.out.print(s: "Melayani Mahasiswa :");
                        dilayani.tampilkanData();
                    }
                }
            }
        } while (pilihan != 0);
    }
}
```

3. Mahasiswa23.java

```
public class Mahasiswa23 {
    String nim, nama, prodi, kelas;

    public Mahasiswa23(String nim, String nama, String prodi, String kelas) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
        this.kelas = kelas;
    }

    public void tampilkanData() {
        System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);
    }
}
```

Hasilnya

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layanan Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa Dalam Antrian
0. Keluar
Pilih operasi: 1
NIM : 231
Nama : qwe
Prodi : qwe
Kelas : 12d
qwe berhasil masuk antrian

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layanan Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa Dalam Antrian
0. Keluar
Pilih operasi: █
```

3.2.3 Pertanyaan

Lakukan modifikasi program dengan menambahkan method baru bernama LihatAkhir pada class AntrianLayanan yang digunakan untuk mengecek antrian yang berada di posisi belakang. Tambahkan pula daftar menu 6. Cek Antrian paling belakang pada class LayananAkademikSIKAD sehingga method LihatAkhir dapat dipanggil!

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layanan Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa Dalam Antrian
6. Cek Antrian paling belakang
0. Keluar
Pilih operasi: 6
Mahasiswa Paling Belakang :
NIM - NAMA - PRODI - KELAS
12312 - qweqeqw - wqeq - qweqw
```

Tugas

1. AntrianKRSMain23.java

```
import java.util.Scanner;

public class AntrianKRSMain23 {
    private static Scanner sc = new Scanner(System.in);

    Run | Debug
    public static void main(String[] args) {
        System.out.print(s: "Masukkan kapasitas maksimal antrian (max 10): ");
        int kapasitas = sc.nextInt();
        if (kapasitas > 10) kapasitas = 10;
        AntrianKRS23 antrian = new AntrianKRS23(kapasitas);

        int pilihan;
        do {
            System.out.println(x: "\n=== MENU ANTRIAN KRS ===");
            System.out.println(x: "1. Tambah Antrian (Mahasiswa)");
            System.out.println(x: "2. Proses KRS (Panggil 2 mahasiswa)");
            System.out.println(x: "3. Cek Antrian Kosong");
            System.out.println(x: "4. Cek Antrian Penuh");
            System.out.println(x: "5. Kosongkan Antrian");
            System.out.println(x: "6. Tampilkan Semua Antrian");
            System.out.println(x: "7. Tampilkan 2 Antrian Terdepan");
            System.out.println(x: "8. Tampilkan Antrian Paling Akhir");
            System.out.println(x: "9. Cetak Jumlah Antrian");
            System.out.println(x: "10. Cetak Jumlah Sudah Proses KRS");
            System.out.println(x: "11. Cetak Jumlah Belum Proses KRS");
            System.out.println(x: "0. Keluar");
            System.out.print(s: "Pilihan Anda: ");
            pilihan = sc.nextInt();
            sc.nextLine();

            switch (pilihan) {
                case 1:
                    tambahMahasiswa(antrian);
                    break;
                case 2:
                    antrian.prosesKRS();
                    break;
            }
        }
    }
}
```

2. AntrianKRS23.java

```
public class AntrianKRS23 {
    private Mahasiswa[] queue;
    private int front;
    private int rear;
    private int size;
    private int max;
    private int handledCount;
    private final int MAX_HANDLED = 30;

    public AntrianKRS23(int kapasitas) {
        this.max = kapasitas;
        queue = new Mahasiswa[max];
        front = 0;
        rear = -1;
        size = 0;
        handledCount = 0;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public boolean isFull() {
        return size == max;
    }

    public void clear() {
        if (isEmpty()) {
            front = 0;
            rear = -1;
            size = 0;
            System.out.println(x: "Antrian berhasil dikosongkan.");
        } else {
            System.out.println(x: "Antrian sudah kosong.");
        }
    }

    public void tambahAntrian(Mahasiswa m) {
        if (isFull()) {
            System.out.println(x: "Antrian penuh, tidak dapat ditambahkan.");
        } else {
            rear++;
            if (rear == max) rear = 0;
            queue[rear] = m;
            size++;
        }
    }

    public void prosesKRS() {
        if (isEmpty()) {
            System.out.println(x: "Antrian kosong, tidak ada yang diproses.");
        } else {
            System.out.println(x: "Proses KRS dimulai. Panggilan ke-1: ");
            queue[front].prosesKRS();
            front++;
            if (front == max) front = 0;
            size--;
            System.out.println(x: "Panggilan ke-2: ");
            queue[front].prosesKRS();
            front++;
            if (front == max) front = 0;
            size--;
            handledCount++;
        }
    }

    public void tampilkanSemua() {
        if (isEmpty()) {
            System.out.println(x: "Antrian kosong.");
        } else {
            System.out.println(x: "Antrian: ");
            for (int i = 0; i < size; i++) {
                System.out.print(queue[(front + i) % max]);
                if (i < size - 1) System.out.print(", ");
            }
            System.out.println();
        }
    }

    public void tampilkanTerdepan() {
        if (isEmpty()) {
            System.out.println(x: "Antrian kosong.");
        } else {
            System.out.println(x: "Antrian terdepan: ");
            System.out.print(queue[front]);
            System.out.println();
        }
    }

    public void tampilkanPalingAkhir() {
        if (isEmpty()) {
            System.out.println(x: "Antrian kosong.");
        } else {
            System.out.println(x: "Antrian paling akhir: ");
            System.out.print(queue[(front + size - 1) % max]);
            System.out.println();
        }
    }

    public void cetakJumlah() {
        System.out.println(x: "Jumlah antrian: " + size);
    }

    public void cetakJumlahSudahProsesKRS() {
        System.out.println(x: "Jumlah antrian sudah proses KRS: " + handledCount);
    }

    public void cetakJumlahBelumProsesKRS() {
        System.out.println(x: "Jumlah antrian belum proses KRS: " + (size - handledCount));
    }
}
```

3.Mahasiswa.java

```
public class Mahasiswa {  
    private String nim;  
    private String nama;  
    private String prodi;  
    private String kelas;  
  
    public Mahasiswa(String nim, String nama, String prodi, String kelas) {  
        this.nim = nim;  
        this.nama = nama;  
        this.prodi = prodi;  
        this.kelas = kelas;  
    }  
  
    public String getNim() {  
        return nim;  
    }  
  
    public String getNama() {  
        return nama;  
    }  
  
    public String getProdi() {  
        return prodi;  
    }  
  
    public String getKelas() {  
        return kelas;  
    }  
  
    public void tampilkanData() {  
        System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);  
    }  
}
```

Hasil

```
1. Tambah Antrian (Mahasiswa)  
2. Proses KRS (Panggil 2 mahasiswa)  
3. Cek Antrian Kosong  
4. Cek Antrian Penuh  
5. Kosongkan Antrian  
6. Tampilkan Semua Antrian  
7. Tampilkan 2 Antrian Terdepan  
8. Tampilkan Antrian Paling Akhir  
9. Cetak Jumlah Antrian  
10. Cetak Jumlah Sudah Proses KRS  
11. Cetak Jumlah Belum Proses KRS  
0. Keluar  
Pilihan Anda: 1  
Masukkan NIM: 2131  
Masukkan Nama: qweq  
Masukkan Program Studi: wqe  
Masukkan Kelas: 1  
qweq berhasil masuk antrian.
```

Diagram

